Lekcja 5. Preprocesor i dyrektywy - zadania dla ucznia

Czas pracy: 45-90 minut. Zadania wykonuj w C++ w dowolnym środowisku IDE lub w kompilatorze online obsługującym projekty wieloplikowe.

A. Pytania kontrolne

1. 1. Co to jest preprocesor i kiedy działa?
2. 2. Czym dyrektywa preprocesora różni się od zwykłej instrukcji C++?
3. 3. Wyjaśnij różnicę między #include <iostream> a #include "operacje.h".
4. 4. Po co w plikach nagłówkowych stosuje się header guard?
5. 5. Jakie zastosowania ma dyrektywa #define?
6. 6. Dlaczego w C++ do stałych liczbowych często lepiej użyć constexpr niż #define?
7. 7. Co robi dyrektywa #undef?
8. 8. Czym różni się #ifdef od #ifndef?
9. 9. Do czego można wykorzystać kompilację warunkową w wersji DEBUG programu?
10. 10. Wyjaśnij, dlaczego makrofunkcje mogą być mniej bezpieczne niż zwykłe funkcje.
11. 11. Co oznacza skrót IDE?
12. 12. Co oznacza skrót API?

B. Analiza kodu

Zadanie B1. Zapisz, jaki tekst programu otrzyma kompilator po przetworzeniu przez preprocesor.

```
#define PI 3.14
#define PROMIEN 5

double pole = PI * PROMIEN * PROMIEN;
```

Zadanie B2. Wyjaśnij, dlaczego poniższy kod może dać wynik inny niż oczekiwany.

```
#define KWADRAT(x) ((x) * (x))
int a = 3;
int wynik = KWADRAT(a++);
```

Zadanie B3. Określ, który komunikat zostanie dołączony do kompilacji.

```
#define DEBUG
#ifdef DEBUG
std::cout << "Tryb diagnostyczny";
#else
std::cout << "Tryb zwykły";
#endif
```

C. Zadania programistyczne

Zadanie 1 - biblioteki standardowe

Napisz program, który wykorzystuje biblioteki <cmath>, <string> i <iostream>. Program ma wczytać imię użytkownika oraz liczbę, a następnie wypisać pierwiastek z tej liczby oraz komunikat powitalny.

Zadanie 2 - projekt wieloplikowy

Utwórz pliki main.cpp, jednostki.h i jednostki.cpp. W pliku jednostki.h zadeklaruj funkcje kmNaMile, kgNaFunty i litryNaGalony. W pliku jednostki.cpp zdefiniuj te funkcje. W main.cpp przetestuj ich działanie.

Zadanie 3 - makrostałe i constexpr

Napisz dwie wersje programu obliczającego pole koła: pierwszą z #define PI, drugą z constexpr double PI. Porównaj czytelność i bezpieczeństwo obu rozwiązań.

Zadanie 4 - kompilacja warunkowa DEBUG

Napisz program, który w trybie DEBUG wypisuje dodatkowe komunikaty o wykonywanych krokach, a bez DEBUG wypisuje tylko wynik końcowy.

Zadanie 5 - header guard

Utwórz plik matematyka.h z deklaracją funkcji kwadrat. Zabezpiecz plik klasycznym header guard. Następnie dołącz go dwukrotnie w main.cpp i sprawdź, czy projekt nadal się kompiluje.

Zadanie 6 - zadanie rozszerzone

Napisz program obliczający pole i obwód prostokąta. Wymuś w programie warunkową kompilację: jeżeli zdefiniowano makro USE_MACROS, obliczenia mają używać makrofunkcji; w przeciwnym razie zwykłych funkcji.

D. Kryteria sukcesu

- program kompiluje się bez błędów;
- użyte dyrektywy preprocesora są uzasadnione;
- nazwy plików i funkcji są czytelne;
- kod jest podzielony na pliki tam, gdzie wymaga tego treść zadania;
- uczeń potrafi wyjaśnić działanie użytych dyrektyw.